Neuro-Symbolic Causal Inference: Classical Machine Learning and Bayesian Networks

Yug Johri, Vidit Sachan, Aditya Verma, Kaavya Gala, Ishita Thakur
Team T094, Section E
Newton School of Technology, Rishihood University
Phase 1 Evaluation: 2 November 2026

***Abstract*—This Phase 1 project separates three tasks that are often blurred together: predicting whether a causal claim follows from a textual premise, extracting explicitly stated relations, and estimating effects in a supplied causal model. We train classical text models on 12,000 Corr2Cause examples and evaluate after removing exact premise overlap between splits. Logistic regression achieves 0.700 test macro F1, compared with 0.460 for a majority classifier. A conservative lexical extractor preserves sentence-level provenance and requires edge review before graph construction. A separate three-variable synthetic system demonstrates observational and interventional inference with Bayesian parameter estimation. Its two sprinkler interventions differ from their known generating-process values by approximately 0.009 and 0.011. These results support a reproducible classical baseline, rather than a claim of general causal understanding.**

I. Problem and scope

The project investigates how a language-facing component can propose causal structure while a formal graph component enforces structural constraints and computes explicit queries. The title supplied by the course is Neuro-Symbolic Causal Inference via LLM Knowledge Extraction and Bayesian Networks. For this submission, only the Advanced Machine Learning component is implemented. The language-facing model is TF-IDF with classical classifiers; there is no language-model service or neural training. The system has a Streamlit interface and a reproducible notebook. Its outputs distinguish reasoning labels, text mentions, reviewed graph edges, and probabilities from observational data.

II. Literature and methodological choices

Pearl describes causal models and interventions as changes to mechanisms, rather than ordinary conditioning [1]. This distinction motivates deleting incoming edges during an intervention. Jin et al. introduce Corr2Cause to test inference from stated statistical relations to causal conclusions [2]. Its synthetic examples provide a controlled reasoning task, but do not provide annotated scientific sentences for open-domain cause-effect extraction. We therefore do not interpret Corr2Cause classification scores as extraction scores. Scikit-learn documents sparse TF-IDF representations for text learning [3]. They provide a transparent, computationally inexpensive baseline whose feature weights can be inspected. pgmpy supplies discrete Bayesian models, parameter estimation and exact inference [4]. Together, these sources motivate separate evaluation of language classification, graph validation and causal queries. BERT is part of the broader project brief but is not implemented or evaluated in Phase 1.

III. Dataset, split audit and EDA

The source is the official causal-nlp/corr2cause dataset on Hugging Face [5], pinned to revision 42ba12c769e11ff6427c9f52d7db58e3f9bf3e53. To keep the local run bounded, we use the first 12,000 training rows and the complete development and test CSVs. This prefix is not a random sample and may underrepresent the full dataset. Every downloaded slice has a SHA-256 hash in data/raw/manifest.json. A row combines a premise and a hypothesis with a binary entailment label. Before fitting any text features, we normalize and hash the premise, retain the original training split, and remove premise groups already seen in earlier splits. This leaves 12,000 training rows, 1,040 development rows and 1,044 test rows. There are 213, 21 and 18 exact premise groups respectively. Thirty-six development rows and 118 test rows are removed. Exact input duplicates are also removed. The grouping does not guarantee separation of semantically equivalent graphs under variable renaming. The test set contains 888 negatives and 156 positives. This imbalance is why accuracy is not the selection criterion. Saved EDA includes class counts, word-length distributions, missing-input counts and relation-template slices.

IV. Features and classical models

The main representation is word TF-IDF with unigrams and bigrams, a maximum vocabulary of 18,000 features and sublinear term frequency for logistic regression. Single-letter tokens are retained because A, B and C denote variables in this dataset. The vectorizer is fitted inside a pipeline using training text only. Smooth inverse document frequency is $\log((1 + N)/(1 + df)) + 1$, followed by L2 normalization. The score is a sigmoid applied to a linear combination of sparse features. Logistic regression uses C = 1, balanced class weights, a maximum of 1,500 iterations and random seed 42. The alternatives are a most-frequent classifier and multinomial naive Bayes with alpha = 1. A unigram-only logistic model is a prespecified ablation, excluded from selection. Candidate models are selected by development macro F1. Test results are computed after selection and are never used to choose the winner. There is no hyperparameter search or tuning on test

labels. Positive-class precision, recall and F1 accompany accuracy and macro F1. Classifier probabilities are uncalibrated model scores, not causal confidence.

V. Measured reasoning results

The selected TF-IDF plus logistic regression model scores 0.661 development macro F1 and 0.700 test macro F1. Test positive-class precision is 0.392, recall is 0.923 and F1 is 0.551. Accuracy is 0.775. The confusion matrix, with actual labels as rows and predicted labels as columns, is [[665, 223], [12, 144]]. The majority classifier reaches 0.851 accuracy but zero positive-class recall, illustrating the weakness of accuracy on this split. Naive Bayes achieves 0.465 test macro F1. The unigram ablation achieves 0.521, so retaining local variable relationships substantially improves this bounded experiment, while independent validation remains necessary. A seed-42 group bootstrap, resampling the 18 test premise groups with replacement 300 times, yields an approximate 95% interval of [0.670, 0.734] for the selected macro F1. This interval describes this retained test sample; it is not a population guarantee. The model produces 223 false positives and 12 false negatives. Feature-weight and template-error exports support inspection of lexical shortcuts rather than claims that the model learned formal d-separation.

VI. Text extraction and graph review

The extractor supports simple affirmative clauses of the form X causes Y, X leads to Y and X results in Y. A small alias map unifies rain/rainfall, sprinkler/sprinklers and wet grass. Each candidate retains its document identifier, sentence identifier, original sentence and extraction method. Sentences containing explicit uncertainty or negation markers are conservatively skipped. The parser is deliberately narrow: passive constructions, richer syntax, context-dependent entity resolution and implicit mechanisms are unsupported. The interface requires explicit approval for every proposed edge. The graph builder rejects self loops, duplicate edges and edges that would create a directed cycle. It uses source order, so the retained graph is not an optimal or uniquely identified causal graph. Sixteen authored regression documents contain 13 gold directed mentions. The extractor recovers ten and produces no additional mentions: precision 1.000, recall 0.769 and F1 0.870. This is a development regression fixture, not an independent extraction benchmark. A separate four-mention graph fixture retains three correct DAG edges and rejects one cycle-closing edge. Its perfect edge precision and recall establish expected behavior only on that fixture. A held-out annotated scientific/news corpus is still required before claiming open-domain extraction performance.

VII. Bayesian model and interventions

The probability demonstration uses binary Rain (R), Sprinkler (S) and Wet grass (W), with edges R to S, R to W and S to W. These are declared assumptions for a synthetic system, not discoveries from text. The factorization is P(R,S,W) = P(R) P(S|R) P(W|R,S). We generate 6,000 observations with seed 42: P(R=1)=0.3; P(S=1|R=1)=0.05 and P(S=1|R=0)=0.6. Wet-grass probabilities for (R,S)=(0,0),(0,1),(1,0),(1,1) are 0.02, 0.8, 0.9 and 0.99. The strong association between rain and sprinkler use creates confounding. Conditional probability tables are fitted with a BDeu prior of equivalent sample size five. Variable elimination computes queries. For an intervention, the network removes incoming edges to the intervention variable and conditions on its chosen state. The remaining factor for that fixed variable cancels in normalization. For S=0, the observed wet-grass probability is 0.452 and the fitted interventional value is 0.275, against an oracle value of 0.284. For S=1, the corresponding values are 0.801, 0.846 and 0.857. The oracle uses P(W=1|do(S=s)) = sum_r P(W=1|r,s)P(r). Causal interpretation requires the declared graph, observed confounders, positivity and stable mechanisms. The user-reviewed text graph is exported separately and does not silently acquire these synthetic CPDs. Population intervention queries are supported; individual counterfactual inference is not claimed.

VIII. Reproducibility and software validation

The local project includes the raw data slices, source manifest, processed splits, trained estimator, prediction files, measured metrics, diagnostic plots, notebook and scripts. requirements-lock.txt records the installed versions. The notebook runs data inspection, the experiment, graph checks and intervention examples without external model calls. The application binds to 127.0.0.1 and is not deployed. Tests check split separation, class presence, negative/uncertain extraction behavior, graph cycle/self-loop rejection, agreement with known intervention answers, and rendering of all six Streamlit pages plus an inference action. The run uses Python 3.12. Installing the environment and refreshing source data require internet access; the included model and local demonstrations work offline. The report, notebook and interface deliberately preserve the differences between source data, authored fixtures and synthetic observations.

IX. Discussion and conclusion

The classical reasoning baseline improves macro F1 over majority prediction, but its precision remains low and its uncertainty interval is wide. This is consistent with the difficulty of deriving graph-level conclusions from surface text features. The graph stage adds clear structural safeguards and explicit assumptions; it cannot turn an unsupported language prediction into a verified causal relationship. The current implementation is suitable as a transparent Phase 1 baseline and demonstration. Stronger claims require a representative training sample, an independently annotated extraction corpus, domain-specific observed variables and defensible identification assumptions. No fabricated domain validation or individual contribution history is included. Each team member should be able to reproduce the experiment and explain these boundaries during the viva.

References

[1] J. Pearl, Causality, 2nd ed., Cambridge University Press, 2009. https://bayes.cs.ucla.edu/BOOK-2K/

[2] Z. Jin et al., Can Large Language Models Infer Causation from Correlation?, ICLR 2024. https://arxiv.org/abs/2306.05836

[3] Scikit-learn, Feature extraction documentation. https://scikit-learn.org/stable/modules/feature_extraction.html

[4] A. Ankan and J. Textor, pgmpy: A Python Toolkit for Bayesian Networks, JMLR, vol. 25, 2024. https://www.jmlr.org/papers/volume25/23-0487/23-0487.pdf

[5] Causal NLP, Corr2Cause dataset. https://huggingface.co/datasets/causal-nlp/corr2cause